

编 号	题 号	标 题	知 识 点	代 码 作 者
例 3-32	UVa 10870	Recurrences	线性递推关系; Q 矩阵	刘汝佳
例 3-33	POJ 3150	Cellular Automaton	循环矩阵的乘法	刘汝佳
例 3-34	UVa 10828	Back to Kernighan-Ritchie	高斯消元; 马尔可夫过程; 实数域的线性方程组 (有特殊情况)	刘汝佳
例 3-35	UVa 11542	Square	XOR 方程组	刘汝佳

3.7 快速傅里叶变换 (FFT)

快速傅里叶变换是一个很有工程价值的算法, 广泛地应用在音频、图像等数字信号处理程序中。本节给出一些与快速傅里叶变换相关的题目以及实现。

例3-36 【生成函数; FFT】高尔夫机器人 (Golf Bot, SWERC 2014, SPOJ SWERC14C)

给出 N 个整数 $k_1 \sim N$, 以及另外 M 个数字 $d_1 \sim M$, $1 \leq N$, $M \leq 2 \times 10^5$, $1 \leq k_i, d_j \leq 2 \times 10^5$ 。计算有多少个 d_i 可以写成不超过两个整数 k_a, k_b 的和, a, b 可以相同。比如 $k_1 \sim_3 = \{1, 3, 5\}$, $d_1 \sim_6 = \{2, 4, 5, 7, 8, 9\}$ 。则 $d_1=2$ 可以写成 $1+1$, $d_2=4$ 可以写成 $1+3$, $d_3=5$ 可以写成 5 , $d_5=8$ 可以写成 $3+5$ 。而 d_4, d_6 无法用两个 k 来表达。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
using namespace std;
const double EPS = 1e-8, PI = acos(-1);

template <typename T>
struct Point { // x + y*i, 复数
    T x, y;
    Point& init(T _x = 0, T _y = 0) {
        x = _x, y = _y;
        return *this;
    }
    Point(T x = 0, T y = 0) : x(x), y(y) {}
    Point& operator+=(const Point& p) {
        x += p.x, y += p.y;
        return *this;
    }
    Point& operator*=(const Point& p) {
        return init(x * p.x - y * p.y, x * p.y + y * p.x);
    }
};

template <typename T>
Point<T> operator+(const Point<T>& a, const Point<T>& b) {
    return {a.x + b.x, a.y + b.y};
}

template <typename T>
Point<T> operator-(const Point<T>& a, const Point<T>& b) {
    return {a.x - b.x, a.y - b.y};
}

```

```

template <typename T>
Point<T> operator*(const Point<T>& a, const Point<T>& b) {
    return {a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x};
}
typedef Point<double> Cplx; // x + y*i, 复数
bool isPowOf2(int x) { return x && !(x & (x - 1)); }
const int MAXN = 1 << 18; // 262144;

// FFT 和插值运算 FFT 所用的  $(w_n)^k$ 
valarray<Cplx> Epsilon(MAXN * 2), Arti_Epsilon(MAXN * 2);
void rec_fft_impl(valarray<Cplx>& A, int n, int level,
                  const valarray<Cplx>& EP) {
    int m = n / 2;
    if (n == 1) return;
    valarray<Cplx> A0(A[slice(0, m, 2)]), A1(A[slice(1, m, 2)]);
    rec_fft_impl(A0, m, level + 1, EP), rec_fft_impl(A1, m, level + 1, EP);
    _for(k, 0, m) A[k] = A0[k] + EP[k * (1 << level)] * A1[k],
        A[k + m] = A0[k] - EP[k * (1 << level)] * A1[k];
}
// 提前计算所有的  $(w_n)^k$ , 提升递归 FFT 的运行时间, 免得每一层重复计算
void init_fft(int n) {
    double theta = 2.0 * PI / n;
    _for(i, 0, n) {
        Epsilon[i].init(cos(theta * i), sin(theta * i)); //  $(w_n)^i$ 
        Arti_Epsilon[i].init(Epsilon[i].x, -Epsilon[i].y);
    }
}
void idft(valarray<Cplx>& A, int n) { // DFT(-1), 从 y 求 a
    rec_fft_impl(A, n, 0, Arti_Epsilon);
    A *= 1.0 / n;
}
void fft(valarray<Cplx>& A, int n) { rec_fft_impl(A, n, 0, Epsilon); }

int main() {
    valarray<int> A(MAXN);
    valarray<Cplx> F(MAXN * 2);
    for (int n, M, x; scanf("%d", &n) == 1 && n;) {
        int N = 1;
        _for(i, 0, n) {
            scanf("%d", &(A[i]));
            while (A[i] * 2 > N) N *= 2;
        }
        _for(i, 0, N) F[i].init();
        F[0].x = 1;
        _for(i, 0, n) F[A[i]] = 1;
        init_fft(N);
        fft(F, N), F *= F, idft(F, N);
        int ans = 0;
        scanf("%d", &M);
        _for(i, 0, M) {

```

```

        scanf("%d", &x);
        if (x < N && fabs(F[x].x) > EPS) ans++;
    }
    printf("%d\n", ans);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.8 节例题 31
同类题目：Super Joker II, UVa 12298
*/

```

例 3-37 【分块；计数；FFT】等差数列（Arithmetic Progressions, CodeChef COUNTARI）

给定一个长度为 n ($1 \leq n \leq 10^5$) 的数组 A ($0 \leq A_i \leq 3 \times 10^4$)，问有多少对 (i, j, k) 满足 $i < j < k$ 且 $A_i - A_j = A_j - A_k$ 。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i,a,b) for( int i=(a); i<(b); ++i)
#define _rep(i,a,b) for( int i=(a); i<=(b); ++i)
using namespace std;
typedef long long LL;
const double EPS = 1e-10, PI = acos(-1);

template<typename T>
struct Point { // x + y * i, 复数
    T x, y;
    Point& init(T _x = 0, T _y = 0) { x = _x, y = _y; return *this; }
    Point(T x = 0, T y = 0): x(x), y(y) {}
    Point& operator+=(const Point& p) { x += p.x, y += p.y; return *this; }
    Point& operator*=(const Point& p) { return init(x * p.x - y * p.y, x * p.y +
y * p.x); }
};

template<typename T>
Point<T> operator+(const Point<T>& a, const Point<T>& b)
{ return {a.x + b.x, a.y + b.y}; }
template<typename T>
Point<T> operator-(const Point<T>& a, const Point<T>& b)
{ return {a.x - b.x, a.y - b.y}; }
template<typename T>
Point<T> operator*(const Point<T>& a, const Point<T>& b)
{ return {a.x*b.x - a.y * b.y, a.x*b.y + a.y * b.x}; }
typedef Point<double> Cplx; // x + y * i, 复数

bool isPowOf2(int x) { return x && !(x & (x - 1)); }

const int N2 = 65536, MAXA = 30000, BLK_CNT = 30, MAXN = 100000 + 4;

```

```

valarray<Cplx> Epsilon(N2),Arti_Epsilon(N2); // FFT和插值运算FFT所用的(w_n)^k
void rec_fft_impl(valarray<Cplx>& A, int level, const valarray<Cplx>& EP){
    int n = A.size(), m = n / 2;
    if (n == 1) return;
    valarray<Cplx> A0(A[slice(0, m, 2)]), A1(A[slice(1, m, 2)]);
    rec_fft_impl(A0, level + 1, EP), rec_fft_impl(A1, level + 1, EP);
    _for(k, 0, m)
        A[k] = A0[k] + EP[k * (1 << level)] * A1[k], A[k + m] = A0[k] - EP[k * (1 << level)] * A1[k];
}

void init_fft(int n) { // 提前计算所有的(w_n)^k, 避免重复计算
    double theta = 2.0 * PI / n;
    _for(i, 0, n) {
        Epsilon[i].init(cos(theta * i), sin(theta * i)); // (w_n)^i
        Arti_Epsilon[i].init(Epsilon[i].x, -Epsilon[i].y);
    }
}

void rec_rev_fft(valarray<Cplx>& A) { // DFT^(-1), 从y求a
    rec_fft_impl(A, 0, Arti_Epsilon);
    A *= 1.0 / A.size();
}

void rec_fft(valarray<Cplx>& A) { rec_fft_impl(A, 0, Epsilon); }
valarray<int> A(MAXN);
valarray<Cplx> A1(N2), A2(N2);
valarray<LL> PREV(011, N2), NEXT(011, N2), INSIDE(N2);
const double invN2 = 1.0 / N2;
int main() {
    int N; scanf("%d", &N);
    _for(i, 0, N) scanf("%d", &(A[i])), A[i]--, NEXT[A[i]]++;
    init_fft(N2); // 初始化所有的单位根
    LL ans = 0;
    int BLK_SZ = (N + BLK_CNT - 1) / BLK_CNT; // 每个BLOCK的大小
    _for(bi, 0, BLK_CNT) {
        int L = bi * BLK_SZ, R = min((bi + 1) * BLK_SZ, N);
        _for(i, L, R) NEXT[A[i]]--;
        INSIDE = 0;
        _for(i, L, R) { // 至少两个元素在这个Block内, 且3个元素都不相等
            _for(j, i + 1, R) if (A[j] != A[i]) {
                int AK = 2 * A[i] - A[j];
                if (0 <= AK && AK < MAXA) // 考虑后两个元素是Ai和Aj
                    ans += PREV[AK] + INSIDE[AK];
                AK = 2 * A[j] - A[i];
                if (0 <= AK && AK < MAXA) // 考虑前两个元素是Ai和Aj, 则后一个元素必然在NEXT,
                    ans += NEXT[AK];
            }
            INSIDE[A[i]]++;
        }
    }
}

```

```

    _for(ak, 0, MAXA) { // 3个元素相等=ak的情况
        LL ki = INSIDE[ak];
        ans += ki*(ki-1)/2*(PREV[ak]+NEXT[ak]); // 两个元素在Block内 C(ki,2)*(PREV+NEXT)
        ans += ki*(ki-1)*(ki-2)/6; // 3个元素都在Block内 C(ki,3)
    }

    if (bi > 0 && bi + 1 < BLK_CNT) { // 只有中间元素在当前Block内
        _for(i, 0, N2) A1[i].init(PREV[i]), A2[i].init(NEXT[i]);
        // 卷积计算, 计算分别位于 Prev 和 Next 内的两个和为 2*ak 的情况
        rec_fft(A1), rec_fft(A2), A1 *= A2, rec_rev_fft(A1);
        _for(ak, 0, MAXA) ans += INSIDE[ak] * llrint(A1[2 * ak].x);
    }

    _for(i, L, R) PREV[A[i]]++;
}

printf("%lld\n", ans);
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 2.8 节例题 33
同类题目: Tile Cutting, ACM/ICPC WF 2015, Codeforces Gym 101239J
*/

```

例3-38 【NTT的DFT】多项式求值 (Evaluate the polynomial, CodeChef POLYEVAL)

给出整数系数多项式 $A(x)=a_0+a_1x+a_2x^2+a_3x^3+\cdots+a_nx^n$, 计算这些多项式对于给定的 Q 个不同整数的值, 结果对 $M=786\,433$ 取模后输出, $0\leq a_i, x_j\leq M$, $N, Q\leq 2.5\times 10^5$ 。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
using namespace std;
typedef long long LL;
const int MOD = 786433, K = 1 << 18, w = 1000;
typedef vector<int> IVec;
int add_mod(int a, int b) {
    LL ret = a + b;
    while (ret < 0) ret += MOD;
    return ret % MOD;
}
int mul_mod(int a, int b) { return ((LL)a * b) % MOD; }
int pow_mod(int a, int b) {
    LL ans = 1;
    while (b > 0) {
        if (b & 1) ans = mul_mod(ans, a);
    }
}

```



```

    a = mul_mod(a, a);
    b /= 2;
}
return ans;
}
int getGen(int P) { // 原根
    unordered_set<int> set;
    _for(g, 1, P) {
        set.clear();
        int pm = g;
        _for(ex, 1, P) {
            if (set.count(pm)) break;
            set.insert(pm);
            pm = (pm * g) % P;
        }
        if (set.size() == MOD - 1) { // 找到原根了
            assert(pm == g);
            return g;
        }
    }
    return -1;
}
int eval(const IVec& A, int x) { // 求A(x)
    int ans = 0, cur = 1;
    for (size_t i = 0; i < A.size(); i++)
        ans = add_mod(ans, mul_mod(A[i], cur)), cur = mul_mod(cur, x);
    return ans;
}
IVec slice_vec(const IVec& vec, int start, int step) {
    IVec ans;
    for (size_t i = start; i < vec.size(); i += step) ans.push_back(vec[i]);
    return ans;
}
// 对于多项式A(x), 使用{w^0, w^1, w^(K-1)}做DFT(数论模运算)
IVec NTT(const IVec& A, const IVec& W, int level = 1) {
    int n = W.size() / level, m = n / 2, An = A.size();
    IVec ans(n, 0);
    if (An < 1) return ans;
    if (n <= 2) {
        _for(i, 0, n) ans[i] = eval(A, W[level * i]);
        return ans;
    }
    const IVec &A0 = NTT(slice_vec(A, 0, 2), W, level * 2),
        &A1 = NTT(slice_vec(A, 1, 2), W, level * 2);
    _for(i, 0, n) ans[i] = add_mod(A0[i % m], mul_mod(W[level * i], A1[i % m]));
    return ans;
}

int main() {

```

```

const int g = getGen(MOD);
int n;
cin >> n;
n++;
IVec A(n), ans(MOD), W(K), B(n);
_for(i, 0, n) cin >> A[i];
_for(i, 0, K) W[i] = pow_mod(w, i);
_rep(a, 0, 2) {
    _for(i, 0, n) B[i] = mul_mod(A[i], pow_mod(g, a * i));
    const IVec& Y = NTT(B, W);
    _for(i, 0, K) ans[mul_mod(pow_mod(g, a), W[i])] = Y[i];
}
ans[0] = A[0];
int Q, x;
cin >> Q;
while (Q--) cin >> x, cout << ans[x] << endl;
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.8 节例题 34
注意：原根的计算可以离线完成，然后在代码中硬编码
同类题目：Sequence, HDU 6589
          序列统计, SDOI 2015, 牛客 NC 20373
          多项式的运算, SCOI 2013, 牛客 NC 20291
*/

```

例 3-39 【NTT；混合基 FFT】性能优化（Optimize, CTSC 2010, 牛客 NC 208302）

给出两个长度为 n 的整数序列 $a[0, \dots, n-1]$, $b[0, \dots, n-1]$ 和非负整数 C 。对于两个序列，定义 “*” 运算，结果为一个长度为 n 的整数序列。例如， $f * g = h$ ，则有 $h_k = \sum_{i+j \equiv k \pmod n} f_i \cdot g_j$ 。

求 $a * b * b * \dots * b$ 每一位模 $(n+1)$ 的值，其中有 C 个 “*” 运算， $(n+1)$ 是质数， n 的质因数大小均不超过 10， $n \leq 5 \times 10^5$, $a[i], b[i]$, $C \leq 10^9$ 。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
typedef long long LL;
typedef vector<int> IVec;
LL MOD;
template <typename T>
T mul_mod(T a, T b) { return (((LL)a) * b) % MOD; }
template <typename T>
T pow_mod(T a, T b) {
    LL ans = 1;
    while (b > 0) {
        if (b & 1) ans = mul_mod((T)ans, a);
```

```

    a = mul_mod(a, a), b /= 2;
}
return ans;
}
const int MAXN = 500000;
const vector<int> Primes = {7, 5, 3, 2};
int getGen(int P) { // 原根
    set<int> set;
    _for(g, 1, P) {
        set.clear();
        int pm = g;
        _for(ex, 1, P) {
            if (set.count(pm)) break;
            set.insert(pm), pm = (pm * g) % P;
        }
        if (set.size() == P - 1) { // 找到原根了
            assert(pm == g);
            return g;
        }
    }
    return -1;
}
IVec vslice(const IVec& A, int start, int step, int count) {
    assert(start >= 0);
    assert(step > 0);
    IVec ans;
    for (size_t i = max(start, 0); i < A.size() && count > 0; i += step, count--)
        ans.push_back(A[i]);
    return ans;
}
// F[i] -> F[w_n^(i), i = 0->n]
void fft_impl(IVec& F, int w, const IVec& D, int level = 0) {
    int n = F.size();
    if (n <= 1) return;
    int p = D[level], m = n / p, wp = pow_mod(w, p), wi = 1;
    vector<IVec> frs;
    _for(r, 0, p) {
        frs.push_back(vslice(F, r, p, m));
        fft_impl(frs.back(), wp, D, level + 1);
    }

    _for(i, 0, n) {
        F[i] = 0;
        _for(r, 0, p) (F[i] += mul_mod(pow_mod(wi, r), frs[r][i % m])) %= MOD;
        wi = mul_mod(wi, w);
    }
}

void FFT(IVec& A, int gen, const IVec& D) { fft_impl(A, gen, D); }

```

```

void IDFT(IVec& A, int gen, const IVec& D) {
    int n = A.size(), nRev = pow_mod(n, n - 1);
    fft_impl(A, pow_mod(gen, n - 1), D);
    _for(i, 0, A.size()) A[i] = mul_mod(A[i], nRev);
}

int main() {
    IVec A(MAXN), B(MAXN), C(MAXN), D;
    for (int n, cn, x; cin >> n >> cn;) {
        cn = cn % n, MOD = n + 1;
        A.clear(), B.clear();
        _for(i, 0, n) cin >> x, A.push_back(x);
        _for(i, 0, n) cin >> x, B.push_back(x);
        D.clear();
        int tmp = n;
        _for(i, 0, Primes.size()) {
            while (tmp % Primes[i] == 0) D.push_back(Primes[i]), tmp /= Primes[i];
        }
        int G = getGen(MOD); // 原根计算
        FFT(A, G, D), FFT(B, G, D);
        C.resize(n);
        _for(i, 0, n) C[i] = (B[i] == 0) ? 0 : mul_mod(A[i], pow_mod(B[i], cn));
        IDFT(C, G, D);
        _for(i, 0, n) printf("%d\n", C[i]);
    }

    return 0;
}

```

例 3-40 【FWT】异或路径 (XOR Path, ACM/ICPC, Asia-Dhaka 2017, Uva 13277)

给出一棵包含 n ($n \leq 10^6$) 个结点的带边权无根树，结点从 1 开始编号，边权均为区间 $[0, 2^{16})$ 内的整数。对于树上两点 u, v ，定义 $d(u, v)$ 为 u 到 v 路径上边权的异或值。对于每个 $x \in [0, 2^{16})$ ，求出使得 $d(u, v) = x$ 的路径有多少条。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>

using namespace std;
typedef long long LL;
static const int maxn = 1e5 + 5, N = 1 << 16;
template <typename T = int>
struct FWT {
    void fwt(T A[], int n) {
        for (int d = 1; d < n; d <= 1) {
            for (int i = 0, m = d << 1; i < n; i += m) {
                for (int j = 0; j < d; j++) {
                    T x = A[i + j], y = A[i + j + d];
```

```

        A[i + j] = (x + y), A[i + j + d] = (x - y); // xor
        // A[i+j] = x+y;                          // 如果是 and 运算
        // A[i+j+d] = x+y;                          // 如果是 or 运算
    }
}
}
}
void ufwt(T A[], int n) {
    for (int d = 1; d < n; d <= 1) {
        for (int i = 0, m = d << 1; i < n; i += m) {
            for (int j = 0; j < d; j++) {
                T x = A[i + j], y = A[i + j + d];
                A[i + j] = (x + y) >> 1, A[i + j + d] = (x - y) >> 1; // xor
                // A[i+j] = x-y;                          // 如果是 and 运算
                // A[i+j+d] = y-x;                          // 如果是 or 运算
            }
        }
    }
}

void conv(T a[], T b[], int n) {
    fwt(a, n), fwt(b, n);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i];
    ufwt(a, n);
}

void self_conv(T a[], int n) {
    fwt(a, n);
    for (int i = 0; i < n; i++) a[i] = a[i] * a[i];
    ufwt(a, n);
}
};

struct Edge {
    int v, w;
    Edge(int _v = 0, int _w = 0) : v(_v), w(_w) {}
};

vector<Edge> G[maxn];
FWT<LL> fwt;
LL A[N + 5];
void dfs(int u, int p = -1, int x = 0) {
    A[x]++;
    for (auto &e : G[u]) if (e.v != p) dfs(e.v, u, x ^ e.w);
}

int main() {
    int T; scanf("%d", &T);
    for (int t = 1, n; t <= T; t++) {

```